

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО» (Университет
ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т
о курсовой работе
Тема «Разработка мобильного приложения с новостной лентой»

Обучающийся:
Ребров Сергей Андреевич, К3439

Преподаватель:
Шатинский Григорий Сергеевич

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Архитектура приложения.....	4
1.1 Общая структура проекта.....	4
1.2 Переходы между разделами.....	4
2 Реализация интерфейсов.....	6
2.1 Экран новостной ленты.....	6
2.2 Экран профиля.....	7
2.3 Экран настроек.....	8
3 Интеграция данных.....	9
4 Работа с уведомлениями.....	10
ЗАКЛЮЧЕНИЕ.....	11

ВВЕДЕНИЕ

Курсовая работа посвящена созданию Android-приложения, которое демонстрирует применение современных подходов и инструментов мобильной разработки. В рамках работы реализован клиент мессенджера с основными пользовательскими экранами: лентой сообщений, профилем пользователя и экраном настроек.

Актуальность проекта определяется широким распространением мобильных приложений и высокой востребованностью специалистов, обладающих современными навыками Android-разработки. Применение архитектурных шаблонов, реактивного управления состоянием, асинхронных операций и фоновых процессов является ключевым для создания качественных, масштабируемых и удобных в сопровождении мобильных решений.

Цель курсовой работы заключается в разработке Android-приложения в соответствии с требованиями лабораторных заданий, охватывающих полный цикл разработки — от проектирования интерфейса до интеграции внешних источников данных и организации фоновой обработки информации.

Для достижения поставленной цели были решены следующие задачи:

- анализ требований и настройка проекта в среде Android Studio;
- разработка интерфейса пользователя и организация навигации между экранами;
- интеграция внешнего API и реализация локального хранения данных;
- обеспечение оффлайн-режима работы приложения;
- настройка фоновой синхронизации данных и системы уведомлений.

1 Архитектура приложения

1.1 Общая структура проекта

Проект построен с разделением на несколько логических слоев (рисунок 1): пользовательский интерфейс, управление состоянием и работу с данными. Такой подход обеспечивает удобство чтения кода и упрощает его последующее расширение и сопровождение.

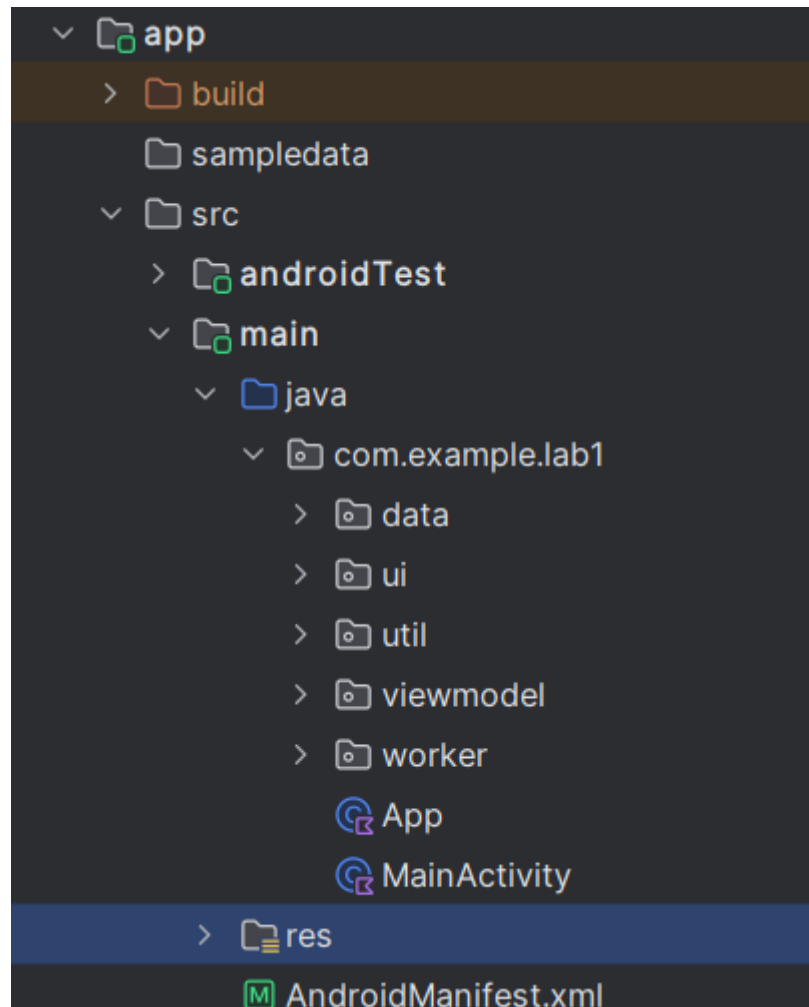


Рисунок 1 — Структура проекта

1.2 Переходы между разделами

Переходы внутри приложения реализованы с использованием Navigation Component (рисунок 2). Основные разделы включают ленту сообщений, пользовательский профиль и настройки. Для удобства работы с интерфейсом предусмотрена нижняя навигационная панель.

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    android:label="@string/news"
    app:startDestination="@id/newsFragment">

    <fragment
        android:id="@+id/newsFragment"
        android:name="com.example.lab1.ui.NewsFragment"
        android:label="@string/news" />

    <fragment
        android:id="@+id/profileFragment"
        android:name="com.example.lab1.ui.ProfileFragment"
        android:label="@string/profile" />

    <fragment
        android:id="@+id/settingsFragment"
        android:name="com.example.lab1.ui.SettingsFragment"
        android:label="@string/settings" />
</navigation>
```

Рисунок 2 — Navigation Component

2 Реализация интерфейсов

2.1 Экран новостной ленты

Главным разделом приложения выступает экран ленты новостей (рисунок 3). На нём отображается перечень сообщений, получаемых из внешнего API или локальной базы данных. Пользователь может просматривать содержимое сообщений и отмечать их в избранное.

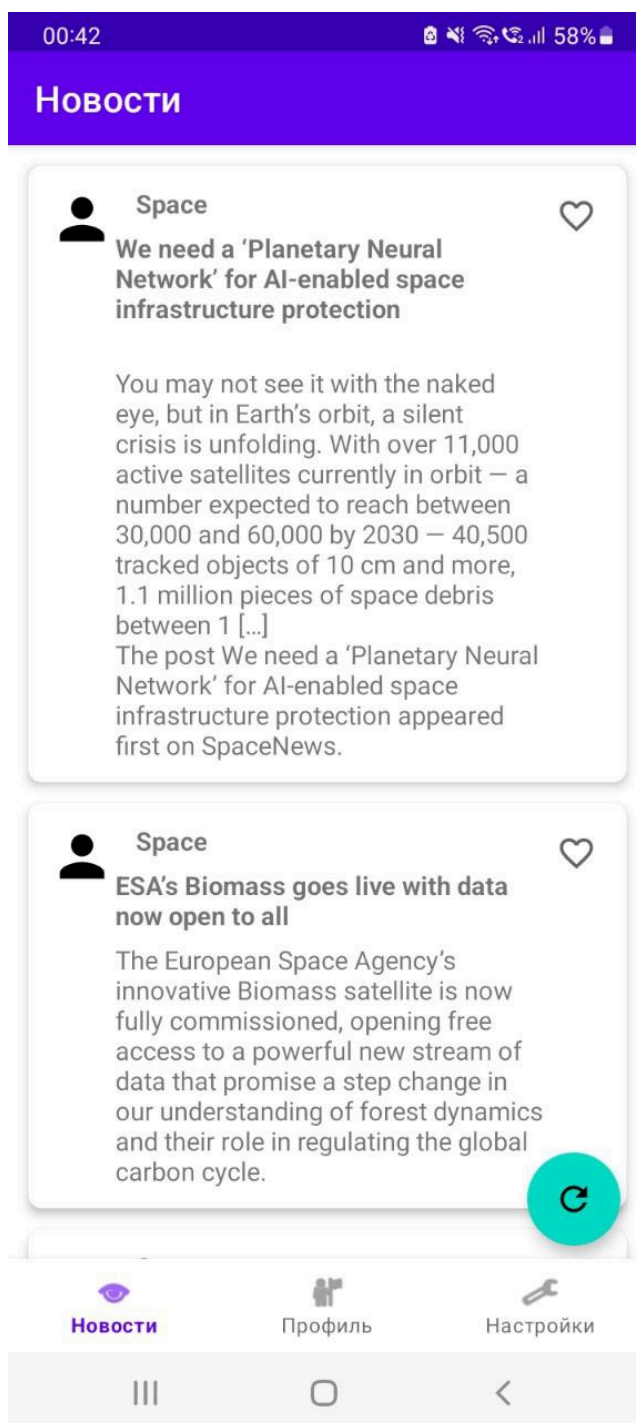


Рисунок 3 — Интерфейс новостной ленты

2.2 Экран профиля

Экран профиля (рисунок 4) служит для просмотра и редактирования информации о пользователе, включая имя и статус. Все данные сохраняются во ViewModel, что гарантирует их сохранность при изменениях конфигурации устройства.



Рисунок 4 — Интерфейс профиля

2.3 Экран настроек

Экран настроек (рисунок 5) позволяет пользователю изменять тему интерфейса. Сделанный выбор сохраняется при последующих запусках приложения.



Рисунок 5 — Интерфейс настроек

3 Интеграция данных

Для получения данных приложение использует публичный API JSON Placeholder (рисунок 6). Запросы к сети осуществляются через библиотеку Retrofit с использованием корутин, что обеспечивает асинхронное выполнение операций и не блокирует основной поток приложения.

```
package com.example.lab1.data.remote

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

2 Usages
object RetrofitClient {
    1 Usage
    private const val BASE_URL = "https://api.spaceflightnewsapi.net/v4/"

    val api: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(baseUrl = BASE_URL)
            .addConverterFactory(factory = GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}
```

Рисунок 6 — Интеграция с API

4 Работа с уведомлениями

Фоновое обновление данных выполняется с помощью SyncWorker (рисунок 7). Периодические задачи обновляют содержимое приложения и информируют пользователя о появлении новых сообщений. Для правильной работы уведомлений на современных версиях Android реализован запрос необходимых разрешений во время работы приложения.

```
package com.example.lab1.worker

import android.content.Context
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import com.example.lab1.data.MessageRepository
import com.example.lab1.util.NotificationHelper

2 Usages
class SyncWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker( appContext = ctx, params) {
    1 Usage
    private val repo = MessageRepository(ctx.applicationContext)

    override suspend fun doWork(): Result {
        return try {
            val (_, isNew) = repo.fetchAndSaveWithFlag()
            if (isNew) {
                NotificationHelper.showNotification(applicationContext, text = "Новые данные получены")
            }
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}
```

Рисунок 7 — Фоновое обновление данных

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано Android-приложение, демонстрирующее современные подходы к созданию мобильных приложений. Реализованы ключевые функциональные экраны — лента сообщений, профиль пользователя и настройки — с использованием архитектуры MVVM, реактивного управления состоянием через LiveData, а также работы с внешним API и локальной базой данных для обеспечения оффлайн-доступа. Дополнительно реализованы фоновая синхронизация данных и система уведомлений, что позволяет поддерживать актуальность информации и улучшает взаимодействие пользователя с приложением.

Разработанный проект показывает эффективность разделения логики на слои, упрощает поддержку и расширение функционала, а также служит практическим примером интеграции сетевых запросов, локального хранения и фоновых процессов в Android-приложении. Выполненная работа позволяет закрепить знания по архитектурным шаблонам, современным инструментам Android-разработки и подготовить базу для дальнейшего развития функционала приложения.